

Answer ALL questions, there is only one correct option, No Marks for partial Answers, no negative marking. Assume all header file included

Admission Number and Section: _____

Name: _____

1. What is the output?

```
int func(int a)
{
    a++;
    return (a);
}
int main()
{
    int i = 10;
    i = func(func( i));
    printf ("%d",i);
    return 0;
}
```

A. 10 B. 11 **C. 12** D. 13

2. What is the output?

```
int main( )
{
    char Str1[]="Today we have QUIZ-II Test" ;
    char Str2[]="Today we have QUIZ-II Test ";
    if(Str1==Str2)
        printf("I am correct");
    else
        printf("I am not correct");
    return 0;
}
```

A. I am correct **B. I am not correct**
C. Compilation Error D. None of these

3. What is the output?

```
void func(int *x, int *y)
{
    *x = (*x + *y) * (*x = *y);
    return;
}
int main( )
{
    int a=20, b=10;
    func(&a, &b);
    printf ( "%d %d \n", a, b );
    return 0;
}
```

A. 300 10 B. 30 1
C. 600 20 D. Compilation Error

4. What is the output?

```
int func(int a)
{
    if(a>1)
        return func(--a) + 10 - a++;
    else
        return 0;
}
int main()
{
    int a = 2;
    printf("%d", func(a));
    return 0;
}
```

A. 8 **B. 9** C. 16 D. Compilation Error

5. What is the index of the last element in an array arr of size n?

A. n **B. n-1** C. n+1 D. 2n

6. Which operator is used to access the value pointed to by a pointer?

A. * B. & C. -> D. #

7. What is the role of recursion in functions?

A. To make the function more complex
B. To call itself within its definition
C. To reduce the efficiency of the program
D. To avoid using loops in programming

8. Which of the following best describes the relationship between arrays, functions, and pointers?

A. Arrays are stored in functions, and pointers access them
B. Functions return pointers to arrays for manipulation
C. Arrays are often passed to functions using pointers
D. Pointers are never used with arrays in C

9. What is the output?

```
int main() {
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int *p = &arr[0][0];

    printf("%d %d %d\n", *p, *(p+5), *(p+7));
}
```

```
printf("%d %d %d\n", p[2], p[4],
      p[7]);

return 0;
}
```

- | | |
|-----------------|----------|
| A. 1 5 9 | C. 1 6 9 |
| 3 6 8 | 5 6 3 |
| B. 1 6 8 | D. 1 6 8 |
| 3 5 8 | 3 4 9 |

10. Consider the following C code:

```
int main() {
    unsigned char a = 8;
    unsigned char b = 12;
    unsigned char result = a ^ b;
    printf("%d\n", result);
    return 0;
}
```

What is the output?

- A. 8 **B. 4** C. 12 D. 20

11. What is the output?

```
int recursiveSum(int n) {
    if (n <= 0) {
        return 0;
    }
    else {
        return n + recursiveSum(n - 1);
    }
}

int main() {
    int result = recursiveSum(4);
    printf("Result: %d\n", result);
    return 0;
}
```

What will be the output of this program?

- A. Result: 0 B. Result: 4
C. Result: 10 D. Infinite loop

12. What is the output?

```
void test(int x) {
    x = 10;
}

int main() {
    int a = 5;
    test(a);
    printf("%d", a);
    return 0;
}
```

```
}
```

- A. 5** B. Garbage value
C. 10 D. Compilation error

13. What is the output?

```
void mystery(int n) {
    if (n <= 0)
        return;
    printf("%d ", n);
    mystery(n - 2);
    printf("%d ", n);
}

int main() {
    mystery(5);
    return 0;
}
```

- A. 5 3 1 1 3 5 5 B. 5 3 1 3 5
C. 1 3 5 5 3 1 **D. 5 3 1 1 3 5**

14. What is the output?

```
int modify(int *p) {
    return((*p)++ + 10);
}

int main() {
    int a = 10;
    a = modify(&a);
    printf("%d\n", a);
    return 0;
}
```

- A. Garbage value B. 21 **C. 20** D. 11

15. What is the output?

```
int main() {
    int arr[] = {10, 20, 30, 40};
    int *ptr = arr;
    printf("%d\n", *(++ptr + 2));
    return 0;
}
```

- A. 20 B. 30 **C. 40** D. Garbage value

16. What is the output?

```
int main()
{
    unsigned char a, b;
    a = 0b10101010;
    b = ~a;
    printf("a = %d, ", a);
    printf("b = %d", b);
}
```

```

    return 0;
}

```

- A. a = 85, b = 170
- B. a = 170, b = 85**
- C. a = 180, b = 85
- D. None of these

17. What is the output?

```

int main() {
    int arr[5] = {6, 12, 18, 24, 30};
    int *p = arr;

    printf("%d ", *++p);
    printf("%d ", *p++);
    printf("%d ", *p);
    return 0;
}

```

19. What is the output?

```

int main()
{
    int temp[] = {32, 29, 27, 46, 22, 25, 38, 45, 76, 53};
    int i, j, n = sizeof(temp)/sizeof(int);
    for(i = 0; i < n; ++i)
        for(j = 0; j < i; ++j)
            if (temp[i] > temp[j])
            {
                temp[i] = temp[i] + temp[j];
                temp[j] = temp[i] - temp[j];
                temp[i] = temp[i] - temp[j];
            }
    printf("%d %d", temp[0], temp[n-1]);
    return 0;
}

```

- A. 32 53 B. 53 32 C. 22 76 **D. 76 22**

20. What is the output of the following C program?

```

// Assume sizeof(int) is 4 and sizeof(int*) is 8 on this system
void processArray(int arr[]) {
    printf("Size inside function: %d\n", sizeof(arr));
}

int main() {
    int data[5] = {1, 2, 3, 4, 5};
    printf("Size outside function: %d\n", sizeof(data));
    processArray(data);
    return 0;
}

```

```

}

```

- A. 12 18 18 B. 6 12 18
- C. 12 12 18** D. 6 18 18

18. What is the output?

```

int main() {
    int arr[5] = {6, 12, 18, 24, 30};
    int *p = &arr[4];
    printf("%d ", p[-1]);
    printf("%d ", *--p);
    printf("%d ", *p--);
    printf("%d\n", *p);
    return 0;
}

```

- A. 24 24 30 24 B. 30 30 30 24
- C. 24 18 18 12 **D. 24 24 24 18**

Choose the most likely output based on the assumptions provided in the code comments.

- | | |
|---|--|
| A. Size outside function: 20
Size inside function: 20 | C. Size outside function: 8
Size inside function: 8 |
| B. Size outside function: 20
Size inside function: 8 | D. Size outside function: 5
Size inside function: 4 |

Rough Work